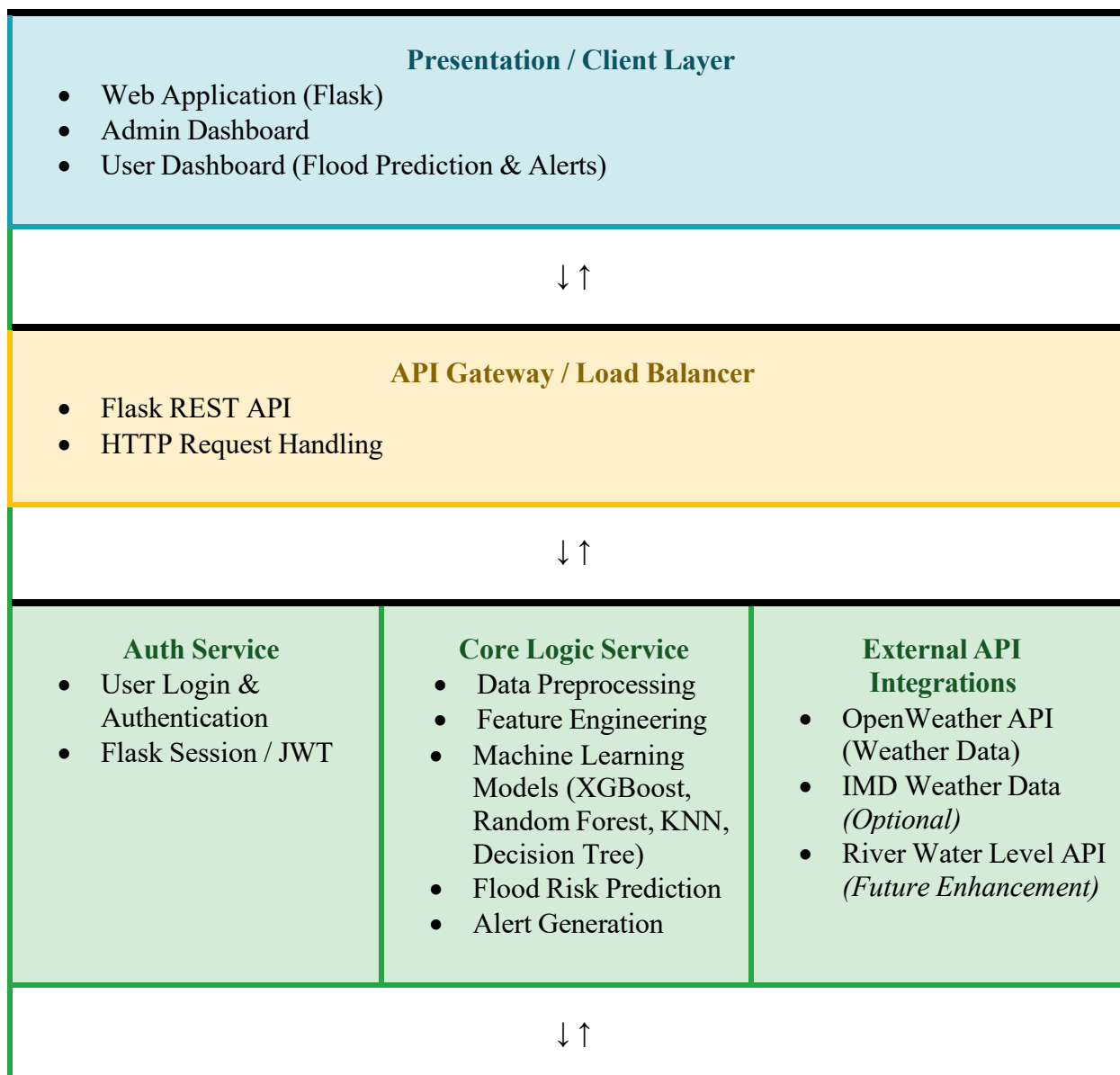


Solution Architecture

Date	05 July 2026
Team ID	
Project Name	Rising Waters
Maximum Marks	5 Marks

Solution Architecture Diagram:



Data / Storage Layer

- Kaggle Historical Flood Dataset (CSV)
- SQLite Database
- Trained ML Models (.pkl)
- Prediction Logs

Component Description Table

Component Name	Description / Role in Architecture	Technologies Used
Presentation Layer	Provides user interface for flood prediction, alerts, and dashboard.	Flask, HTML, CSS, JavaScript
API Gateway	Receives user requests and connects the frontend with ML services.	Flask REST API
Authentication Service	Manages secure user login and access.	Flask Session / JWT
Core Logic Service	Processes data, runs ML models, predicts flood risk, and generates alerts.	Python, Scikit-learn, XGBoost, Pandas, NumPy
External APIs	Retrieves weather and environmental information.	OpenWeather API, IMD API (Optional)
Database / Storage	Stores datasets, trained models, user data, and prediction history.	SQLite, CSV Files, Pickle (.pkl)

Architectural Overview & Technical Description

The solution architecture for the **"Rising Waters"** flood prediction system follows a robust, decoupled **3-Tier Architecture Pattern** (Presentation, Core Logic, and Data Storage). This layered design ensures high availability, modularity, and smooth data flow, enabling real-time weather analytics to be converted into instant emergency alerts.

1. Multi-Layer Data Flow & Component Breakdown

- **Presentation / Client Layer (Frontend):**
 - Actively serves as the interface for end-users, meteorologists, and disaster relief administrators.
 - Built using **Flask, HTML, CSS, and JavaScript** to provide dynamic, responsive dashboards for inputting meteorological data and rendering flood risk predictions.
- **API Gateway & Authentication Service:**
 - Acts as the secure entry point for the application.
 - The **Flask REST API Gateway** securely routes incoming HTTP traffic, while the

Auth Service (utilizing Flask Session/JWT) validates user credentials before granting access to sensitive operational dashboards.

- **Core Logic Service (The ML Engine):**
 - This is the analytical heart of the application, powered by **Python, Scikit-learn, and XGBoost**.
 - It ingests parameters (such as annual rainfall and cloud visibility), executes feature engineering via **Pandas and NumPy**, feeds the data into the optimized **XGBoost model**, and outputs precise risk classifications.
- **External API Integrations:**
 - Enhances real-time capability by pulling live meteorological streams from the **OpenWeather API** and **IMD API**, ensuring the predictive model operates on live seasonal patterns.
- **Data / Storage Layer (Backend):**
 - Handles complete state management and persistence.
 - **SQLite** handles application data and user logs, while serialized model artifacts (**Pickle .pkl files**) are stored statically to allow the Core Logic Service to make rapid, low-latency prediction inferences.